

ENGR-E 321 Lab 8: Romi Autonomous Vehicle with Line Following, ArUco Road Sign Detection, and Drive-by-Video Interface

Jaiden Medina & Brady Adams

TABLE OF CONTENTS

Title Page	1
Table of Contents	2
Abstract	4
I. Introduction	4
A. Project Objective	4
B. System Requirements	4
C. Project Scope	5
II. System Description	5
A. Overall System Overview	5
B. Manual and Autonomous Operation	5
C. Line Following Functionality	5
D. ArUco Road Sign Detection	5
E. Drive-by-Video Interface	5
F. Operator Feedback Interface	6
III. Software Design	6
A. Software Architecture	6
B. Software Architecture Diagram	6
C. Arduino-Level Control Software	7
D. Raspberry Pi Vision Software	7
E. Curses-Based Operator Interface	7
F. Road Sign Decision Logic	7
IV. Control System Design.....	7
A. PID Line-Following Feedback Loop	7
B. Sensor Input and Line Error.....	8
C. PID Correction.....	8
D. Motor Speed Adjustment.....	8
E. Speed Mode Integration.....	8
V. Hardware and Mechanical Design	8
A. Romi 32U4 Platform	9
B. Reflectance Sensor Array	9
C. Raspberry Pi and Camera	9
D. Camera Mount	9
VI. Testing and Results	10
A. Line Following Performance	10
B. ArUco Detection Performance	10
C. Drive-by-Video Performance	10
D. Operator Interface Feedback	10
E. Video Resolution Testing	10
VII. Discussion and Limitations	10
VIII. Future Improvements	10
IX. Conclusion	11

ENGR-E 321 Lab 7: Drive-by-Video Camera Driver

Jaiden Francisco Medina
*Computer Engineering, Luddy School of Informatics,
 Computing, and Engineering*
 Indiana University Bloomington
 Bloomington, IN, USA
 jfmedina@iu.edu

Brady Michael Adams
*Computer Engineering, Luddy School of Informatics,
 Computing, and Engineering*
 Indiana University Bloomington
 Bloomington, IN, USA
 brmadam@iu.edu

Abstract—This project presents the design and implementation of a Romi-based autonomous vehicle system capable of manual operation, autonomous line following, visual road sign detection, speed control, and drive-by-video feedback. The system integrates a Pololu Romi 32U4 mobile platform, Raspberry Pi, Raspberry Pi Camera V2.1, reflectance sensor array, custom camera mount, Redis-based video streaming, OpenCV computer vision, and a curses-based operator interface. The robot can be manually driven to a line track, switched into autonomous mode, and controlled through a feedback loop that uses line sensor data to maintain stable path tracking. ArUco markers are used as visual road signs, where marker IDs command the robot to stop, switch to high speed, or switch to low speed. The laptop-side interface provides live video feedback, marker overlay visualization, and real-time robot status information. During demonstration, the system achieved smooth line following, clear high/low speed distinction, stop/resume behavior, and effective user interface feedback, earning a subtotal score of 90/100. Remaining limitations included intermittent video streaming and occasional missed ArUco marker detections. Future improvements include improved video stream stability, multi-frame marker confirmation, camera angle adjustment, and marker-distance estimation for more precise response timing. Overall, the project demonstrates a complete cyber-physical system that combines embedded control, computer vision, networking, mechanical design, and human-machine interaction into one integrated autonomous robotics platform.

Keywords—autonomous vehicle, cyber-physical systems, line following, ArUco markers, computer vision, Raspberry Pi, Romi 32U4, Redis, OpenCV, embedded systems.

I. INTRODUCTION

The purpose of Lab 7 was to expand the Romi robot system by adding a drive-by-video interface. In this lab, the robot could be driven from a laptop using only the live video feed from the Raspberry Pi camera mounted on the robot. This built on the Lab 5 curses driver and the Lab 6 camera mount.

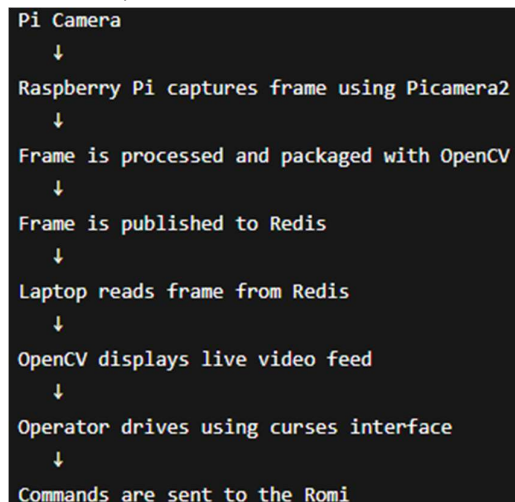
The system used the Raspberry Pi Camera V2.1 to capture video frames, Redis to transmit the frames from the Raspberry Pi, and OpenCV on the laptop to display the live video. The operator could use the curses interface to drive the robot manually, stop the motors, calibrate the sensors, and switch into autonomous mode once the robot reached the line.

II. SYSTEM ARCHITECTURE OVERVIEW

The Lab 7 system was split across three main platforms: the Romi Arduino, the Raspberry Pi, and the laptop. Each platform had a different role. The

Romi handled the physical robot control, the Raspberry Pi handled camera capture and Redis communication, and the laptop handled the operator interface and video display.

A. Overall System Data Flow

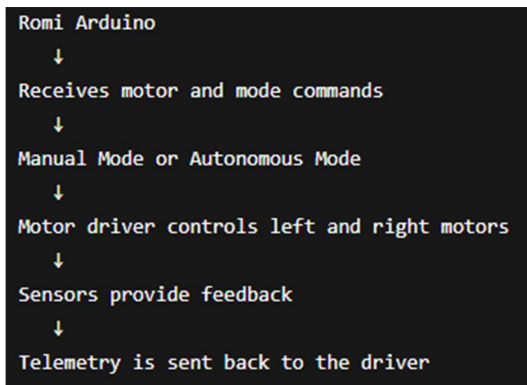


This setup allowed the robot to be driven without directly looking at it. The operator used the camera feed as the main visual feedback source.

III. SYSTEM RUNNING ON THE ROMI ARDUINO

The Romi Arduino was responsible for low-level robot behavior. In manual mode, it applied the motor commands sent from the Python driver. These commands allowed the robot to move forward, reverse, turn left, turn right, and stop.

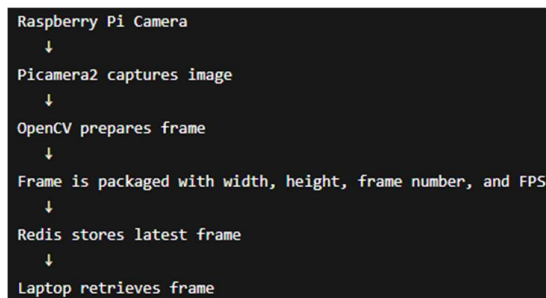
The Romi also supported autonomous mode. Once the robot detected the line and the operator switched modes, the robot used its line sensors and PID control to follow the line. The Arduino also provided sensor readings, encoder values, battery voltage, motor values, and line error data for the laptop interface.



IV. SYSTEM RUNNING ON RASPBERRY PI

The Raspberry Pi acted as the camera and communication system. The `server1.py` program captured video frames from the Raspberry Pi camera using Picamera2. Each frame was stored in Redis with the image data, frame number, resolution, and FPS.

This was important because the lab required the video to be sent through Redis instead of using X Server forwarding. Redis allowed the Raspberry Pi and laptop to communicate over the network while keeping the video display local to the laptop.

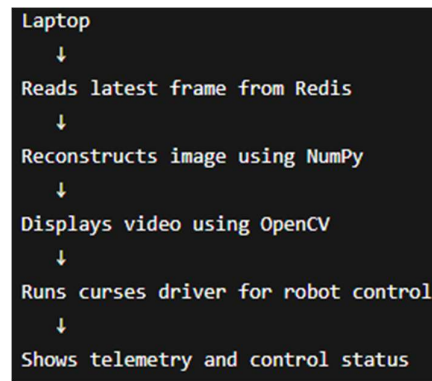


V. SYSTEM RUNNING ON THE LAPTOP

The laptop was the operator station. It ran the OpenCV video client and the curses driver interface.

The video client read the latest frame from Redis, reconstructed the image, and displayed the live camera view. It also showed useful information such as resolution, frame number, server FPS, and client FPS.

The curses interface was used to control the robot. The operator could drive with the arrow keys, stop with the spacebar, switch between manual and autonomous mode, calibrate the sensors, tune PID values, and view telemetry. This made it possible to drive the robot while only using the camera feed.



VI. VIDEO PERFORMANCE TESTING

The lab required testing three different camera resolutions: 640×480 , 320×240 , and 160×120 . Each resolution had a different tradeoff between image quality and frame rate.

Resolution	FPS	Result
640×480	31.57 FPS	Best image quality, but slower response
320×240	43.58 FPS	Best balance of quality and speed
160×120	46.39 FPS	Fastest, but lowest image detail

The best resolution for this system was 320×240 . The 640×480 resolution had the clearest image, but it had the lowest frame rate. The 160×120 resolution had the highest frame rate, but the image was less detailed. The 320×240 resolution was the best choice because it kept the video responsive while still showing enough detail for driving.

VII. RESULTS

The system successfully met the main goals of the lab. The Raspberry Pi captured video from the mounted camera and published the frames through Redis. The laptop read the frames and displayed them using OpenCV. The robot could be driven through the curses interface while the operator watched the live video feed.

The system also supported autonomous mode switching. After manually driving the robot to the line, the sensors could be calibrated and the robot could be placed into autonomous mode. This showed that the video interface worked with the previous manual and autonomous driving system.

VIII. REFLECTION

This lab showed how a cyber-physical system can be split across multiple devices. The Romi handled movement, the Raspberry Pi handled video capture and Redis communication, and the laptop handled the user interface. Keeping these parts separate made the system easier to organize and debug.

The biggest challenge was balancing video quality and responsiveness. A higher resolution made the image clearer, but it also reduced the frame rate. A lower resolution improved speed, but made it harder to see details in the environment. This is why 320×240 was the best practical choice.

This lab also showed how important latency is for remote robot driving. Even if the robot controls work correctly, delayed video can make the system harder to drive. Future improvements could include compressing frames, adding timestamps, and improving synchronization between the video feed and driving commands.

IX. CONCLUSION

Lab 7 successfully added a drive-by-video interface to the Romi robot system. The system used the Raspberry Pi camera, Redis, OpenCV, and a curses-based driver to let the operator drive the robot using only the live video feed. The robot could still be manually controlled and switched into autonomous mode when the line was detected.

Overall, this lab connected embedded control, video streaming, networking, and user interface design into one working robot system. The best video resolution was 320×240 because it provided the best balance between frame rate and image clarity.